



Compiler vs Interpreter

Csa1402-Compiler design

Presented by:
K. Deepthi

Guided By:
Dr N Deepa



I Introduction

- **Compiler** and **Interpreter** are language translators used to convert high-level programming code into machine code.
- A **Compiler** translates the entire program before execution, producing an executable file.
- An **Interpreter** translates and executes the program one statement at a time.
- Both are essential for running programs, but they differ in speed, error handling, and execution process.
- Understanding these differences helps developers choose the appropriate translation method for different applications.



What is a Compiler?

- ❑ A **compiler** is a software program that converts the **entire source code** written in a high-level programming language into **machine code**.
- ❑ It translates the program **before execution** and creates an **executable file**.
- ❑ The compiled program can run directly on the computer without further translation.
- ❑ Compilers improve execution speed because the translation process is completed beforehand.
- ❑ **Examples:** C, C++, Go, Rust
- ❑ **Working Process**
- ❑ **Source Code → Compiler → Machine Code → Program Execution**



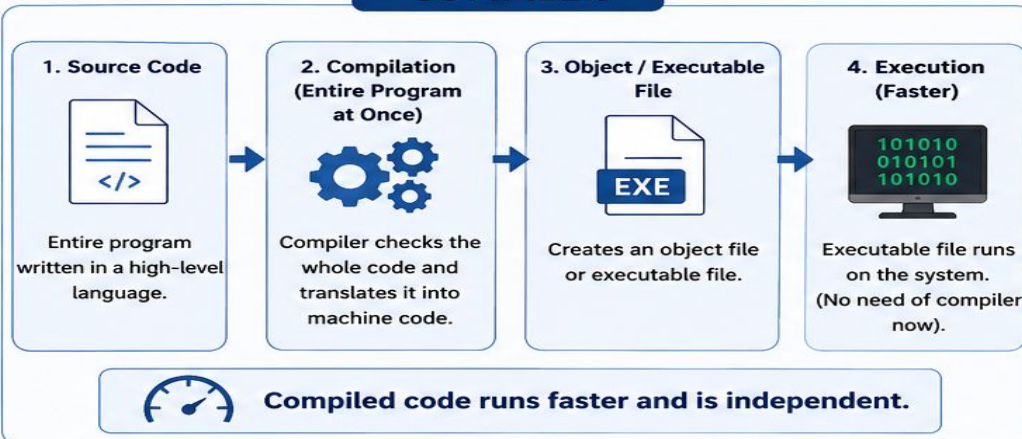
What is an Interpreter?

- ❑ An **interpreter** is a software program that converts high-level programming code into **machine code line by line**.
- ❑ It translates and executes each statement **immediately** without creating a separate executable file.
- ❑ Errors are displayed **as soon as they are encountered**, making debugging easier.
- ❑ Interpreted programs are generally slower because translation happens during execution.
- ❑ **Examples:** Python, JavaScript, Ruby
- ❑ **Working Process**
- ❑ **Source Code → Interpreter → Line-by-Line Translation & Execution**

3. COMPILER vs INTERPRETER

Two ways to translate high-level code into machine language

COMPILER



REAL-LIFE EXAMPLES

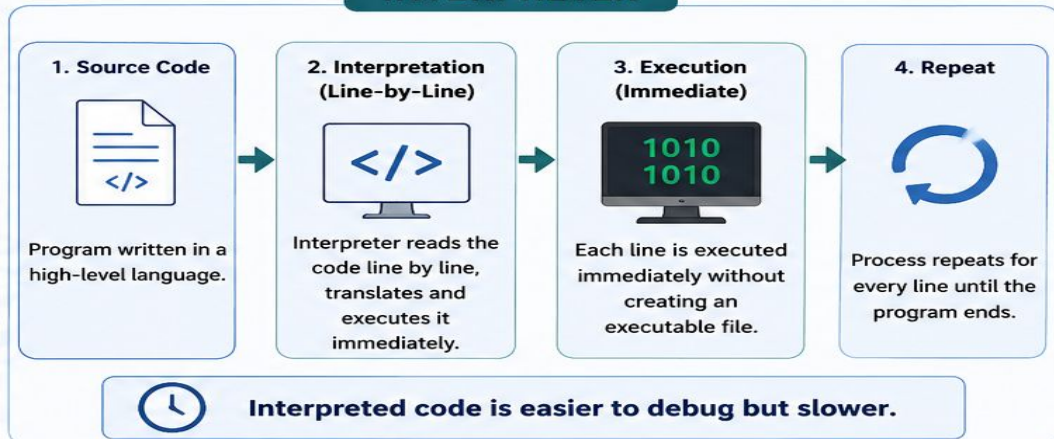


APPLICATIONS



★ Compiled languages are used where performance and efficiency are critical.

INTERPRETER



REAL-LIFE EXAMPLES



APPLICATIONS



★ Interpreted languages are used where flexibility and quick development matter.



Summary: Compiler translates the entire code at once and produces an executable file (faster). Interpreter translates line-by-line and executes immediately (easier to debug).





```
Untitled1.cpp
1  #include <stdio.h>
2
3  int main() {
4      int marks;
5
6      printf("Enter marks: ");
7      scanf("%d", &marks);
8
9      if (marks >= 90)
10         printf("Grade: A");
11     else if (marks >= 75)
12         printf("Grade: B");
13     else if (marks >= 50)
14         printf("Grade: C");
15     else
16         printf("Grade: Fail");
17
18     return 0;
19 }
```

```
C:\Users\sushm\OneDrive\Do
Enter marks: 85
Grade: B
-----
Process exited after 15.38 seconds with return value 0
Press any key to continue . . .
```



main.py



Share

Run

```
1 marks = int(input("Enter marks: "))
2
3 if marks >= 90:
4     print("Grade: A")
5 elif marks >= 75:
6     print("Grade: B")
7 elif marks >= 50:
8     print("Grade: C")
9 else:
10    print("Grade: Fail")
```

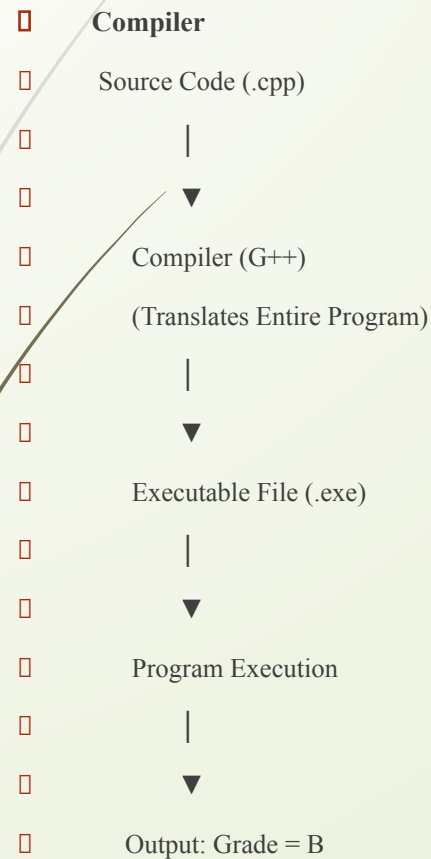
Output

Enter marks: 85
Grade: B

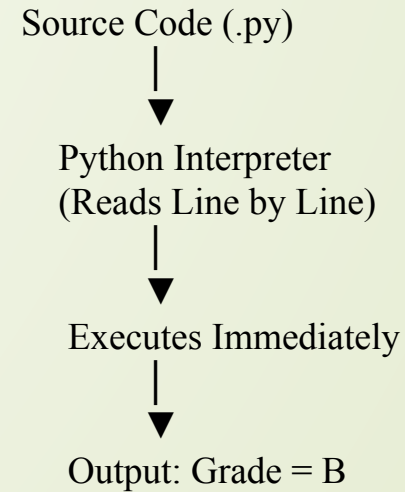
=== Code Execution Successful ===



How they works:



INTERPRETER





Conclusion

- ❑ Compiler translates the whole program at once.
- ❑ Interpreter executes the program line by line.
- ❑ Compiler is faster, while Interpreter is easier to debug.
- ❑ Both convert high-level code into machine language.



Thank you

